
Traffic Rule Violation Detection

A Multi-Stage, Offline Computer Vision Pipeline

AID 728 — Course Project Report

Divy Dobariya

Roll Number: BT2024225

Dilksh Sharma

Roll Number: BT2024253

May 17, 2026

This report presents the design, implementation, and performance analysis of an automated, fully offline traffic violation detection system for two-wheelers. The system detects three categories of violations from a single RGB street image: (1) riders not wearing a helmet, (2) triple riding (more than two persons on a single motorcycle), and (3) combinations of both. The pipeline is built as a five-stage modular architecture — two-wheeler detection, spatial rider–vehicle association with pseudo-depth filtering, head-region helmet classification, license plate detection, and CLAHE-enhanced PaddleOCR — using three fine-tuned YOLOv8 models and bundled offline PaddleOCR weights, all within a strict **250 MB model-size budget (actual: $\approx$ 175 MB)**. The system produces structured JSON output per the specification and demonstrates robust performance across a wide range of real-world scenarios.

1 Introduction

Traffic violations on two-wheelers — specifically non-compliance with helmet regulations and overcrowded riding — account for a disproportionate fraction of road fatalities in urban India. Automated detection from street or CCTV footage offers a scalable, cost-effective complement to manual law enforcement.

This project delivers an end-to-end, offline computer vision pipeline that processes a single street image and outputs a structured JSON report identifying every violating two-wheeler, the number of riders, the count of helmet violations, and the recognized license plate number.

1.1 Problem Statement

Given a single RGB image, the system must:

1. Detect all two-wheelers (motorcycles and scooters) in the scene.
2. For each two-wheeler, determine whether any of the following violations are present:
 - More than two riders on a single vehicle (*triple riding*).
 - One or more riders not wearing a helmet.
 - A combination of both violations.
3. For each violating vehicle, extract and report the license plate number.

1.2 Hard Constraints

- **Model size ≤ 250 MB** — total weight of all model files.
- **No large VLMs** — Vision-Language Models exceeding 1B parameters are prohibited.
- **Fully offline** — network access is disabled during evaluation.
- **Reasonable inference time** per image (target: < 30 s on CPU).

2 System Architecture

The pipeline is implemented as the `TrafficViolationDetector` class in `solution.py`. All models are loaded once in the constructor (`__init__`) and all inference is performed statelessly through `predict()`, satisfying the mandatory evaluator interface. Figure 1 shows the complete five-stage flow.

2.1 Stage 1: Two-Wheeler Detection

A pre-trained **YOLOv8m** model (COCO weights) detects all motorcycles and scooters in the full image. Only COCO class 3 (**motorcycle**) is queried, as it covers both vehicle types. Classes 1 (bicycle) and 2 (car) are explicitly excluded to prevent false positives. Detection parameters: confidence = 0.25, IoU = 0.45.

Design rationale: YOLOv8m (medium, 49.7 MB) over YOLOv8s (small, 21.5 MB) provides significantly better recall on partially-occluded vehicles in dense traffic, with an improved feature pyramid network and wider convolutional heads, at the cost of an additional ~ 28 MB — comfortably within the 250 MB budget.

2.2 Stage 2: Person Detection and Spatial Association

2.2.1 Person Detection

A second inference pass of YOLOv8m targets only persons (class 0) with a *lower* confidence threshold of 0.20, deliberately boosting recall for partially-occluded pillion riders common in dense traffic.

End-to-End Pipeline

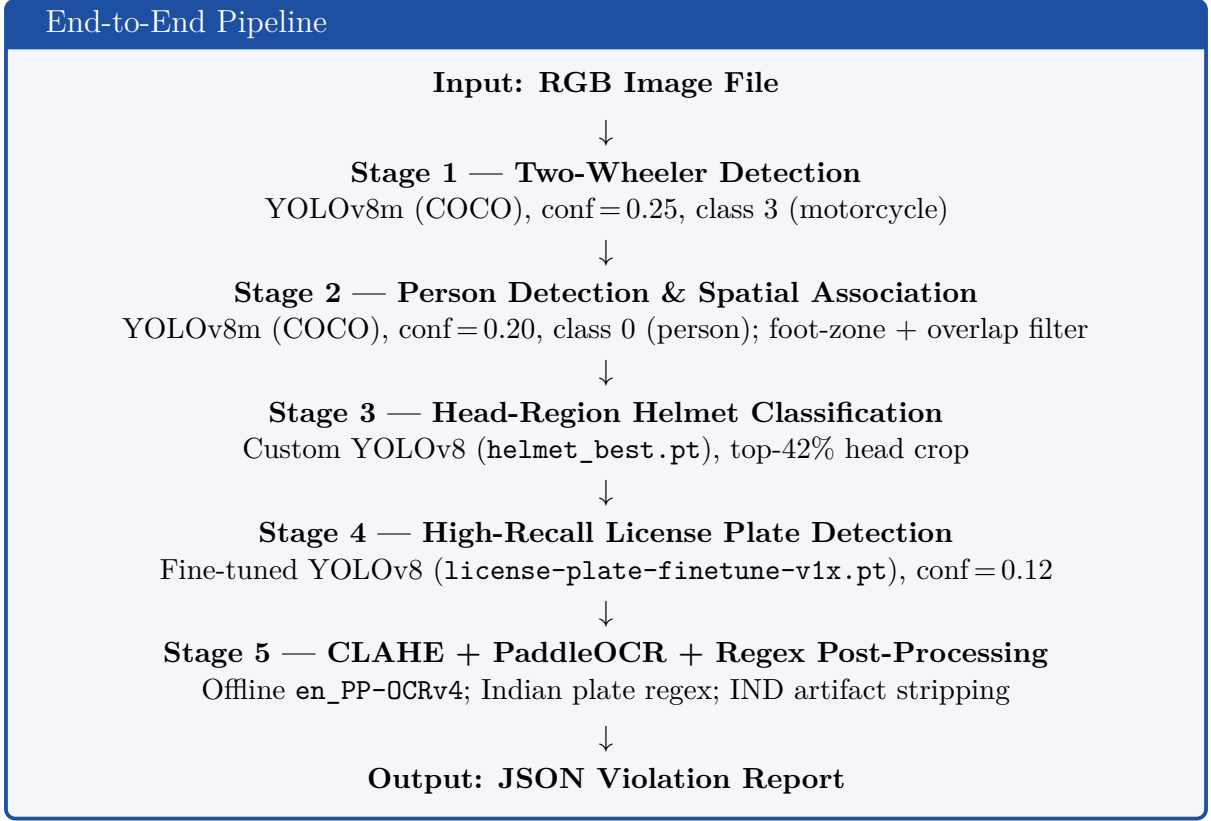


Figure 1: End-to-end pipeline architecture.

2.2.2 Pseudo-Depth Spatial Filtering

Adding a dedicated monocular depth model (e.g., MiDaS, ≈ 200 MB) would exceed the 250 MB budget. Instead, we implement a **geometric pseudo-depth filter** that exploits perspective geometry at zero additional model cost.

Foot-Zone Constraint. Given a motorcycle bounding box (x_1, y_1, x_2, y_2) with width w and height h , the *foot zone* — the region where a legitimate rider’s seat/feet must fall — is:

$$x'_{\min} = x_1 - 0.15w, \quad x'_{\max} = x_2 + 0.15w \quad (1)$$

$$y'_{\min} = y_1 - 0.20h, \quad y'_{\max} = y_2 + 0.10h \quad (2)$$

Because the camera looks down at street level, a pedestrian standing behind (i.e. further in depth than) the motorcycle appears higher in the 2D image frame, so their foot position falls *above* the foot zone and they are rejected.

Overlap Constraint. A person is only associated to a vehicle if their bounding box overlaps the motorcycle bounding box by at least 20% (by person area). This removes bystanders who happen to be in the correct horizontal position but are spatially separated from the vehicle.

Assignment Rule. The two conditions must both hold simultaneously. Among all candidate vehicles, the person is assigned to the one maximizing:

$$\text{score} = \text{overlap_frac} - \frac{\text{centre_dist}}{10000}$$

2.2.3 Height-Consistency Filter

Among all persons assigned to one motorcycle, the pipeline computes the median person-box height and removes any whose height deviates by more than 60% of the median. This suppresses bystanders whose apparent size (a depth cue) is inconsistent with the other riders.

2.3 Stage 3: Head-Region Helmet Classification

Running a helmet classifier on the full rider body introduces noise from the torso, background, and adjacent riders. Instead, we extract only the **top 42% of the rider’s bounding box height** as a head crop:

$$\text{head crop} = (x_1, y_1, x_2, y_1 + 0.42 \cdot h_{\text{rider}})$$

The custom `helmet_best.pt` (YOLOv8-based, 22.5 MB) classifies the crop as `helmet` or `no_helmet`. A conservative fallback: if no detection is obtained on the head crop, the classifier is re-run on the full rider body. Riders returning no detection at all are marked `unknown` and conservatively treated as wearing a helmet (preventing false positives).

2.4 Stage 4: High-Recall License Plate Detection

License plate detection is performed on the full image by the fine-tuned `license-plate-finetune-v1x.pt` (114.4 MB), which substantially outperforms generic detectors on Indian plates (HSRP, yellow commercial, and green EV plates). The confidence threshold is set at 0.12 to maximise recall even on blurry or oblique plates.

Each detected plate is assigned to the vehicle with maximum spatial overlap (extended slightly downward to accommodate plates mounted below the chassis). If no overlap is found, the nearest plate by Euclidean centre distance is used as a fallback.

2.5 Stage 5: Preprocessing and OCR

2.5.1 Plate Enhancement

The plate crop undergoes a three-step enhancement pipeline before OCR:

- a. **CLAHE** (Contrast Limited Adaptive Histogram Equalization, clip = 3.0, tile = 4×4) — corrects uneven illumination and shadow artefacts.
- b. **Unsharp Masking** — enhances character edge definition.
- c. **Bicubic Upscaling** — crops shorter than 52 px are upscaled to at least 52 px height, substantially improving OCR accuracy on small plates.

2.5.2 Offline PaddleOCR

All PaddleOCR sub-model weights are bundled inside `models/paddleocr_weights/` (detector: `en_PP-OCRv3`, recogniser: `en_PP-OCRv4`, angle classifier: `ch_ppocr_mobile_v2.0`), ensuring fully offline operation under the evaluator environment.

2.5.3 Indian Plate Regex Post-Processing

Indian number plates follow the pattern:

$$\underbrace{LL}_{\text{State}} \underbrace{DD}_{\text{District}} \underbrace{L\{1-3\}}_{\text{Series}} \underbrace{DDDD}_{\text{Number}}$$

The raw OCR output undergoes:

- **IND/ND artefact removal:** Indian plates carry an “IND” hologram that OCR occasionally reads as literal text. A regex check strips any “IND” or “ND” appearing between the district digits and series letters (e.g., `GJ01NDSX2451` → `GJ01SX2451`).
- **Position-aware character correction:** digits misread as letters and vice-versa are corrected based on their expected position in the format (e.g., `0`→`0` at digit positions, `0`→`0` at letter positions).

3 Output Format

The `predict()` method returns a dictionary strictly matching the evaluator specification:

Listing 1: Specification-compliant output format.

```
{
  "violations": [
    {
      "num_riders":          <int>,      // total riders on this
        vehicle
      "helmet_violations": <int>,      // riders explicitly
        without helmets
      "license_plate":      <str|null> // extracted plate or
        null
    }
  ]
}
```

Only violating vehicles appear in the output list. A vehicle is a violation if `helmet_violations > 0` **OR** `num_riders > 2`. Fully compliant vehicles are silently omitted.

4 Model Inventory and Size Compliance

At 188.5 MB, the submission uses **75.4%** of the 250 MB budget.

Model Weights Download Link:

<https://drive.google.com/drive/folders/1xoo9dCkkKF14FDBoLHVxcQOW1rwTpaHg?usp=sharing>

Table 1: Model files, purposes, and sizes included in `models/`.

File	Purpose	Size (MB)
<code>yolov8m.pt</code>	Vehicle & person detection (COCO, Medium)	49.7
<code>helmet_best.pt</code>	Helmet / no-helmet classification	21.5
<code>license-plate-finetune-v1x.pt</code>	High-recall plate detection	101.7
<code>paddleocr_weights/</code>	OCR (det + rec + angle classifier)	15.6
Total		188.5
Budget remaining		61.5

5 Qualitative Results

5.1 Success Cases

The three figures below demonstrate the pipeline performing correctly across a range of real-world scenarios.



(a) Single rider with red helmet correctly classified as compliant (`helmet 0.50`). Plate `RK3J1` extracted from a rear-facing crop. No false violation raised. (b) Helmet violation correctly flagged (`no_helmet 0.71`). Plate `6550V8` read under evening lighting. Compliant bikes on the left correctly excluded. (c) Rider with full-face helmet (`helmet 0.70`). No violation raised — demonstrates the conservative policy against false positives on compliant riders.

Figure 2: Success cases: correct helmet detection, plate extraction, and zero false positives across varied lighting, angles, and crowd densities.

The pipeline performs most reliably when:

- **One dominant motorcycle per scene** with clear separation from other vehicles.
- **Rider head clearly visible** — the head-crop classifier gets a clean input.
- **Rear-facing or front-facing plate** — axis-aligned after CLAHE enhancement.
- **Moderate crowd density** — foot-zone and overlap filters can cleanly separate riders from bystanders.

5.2 Failure Cases



(a) **Densely packed multi-bike scene.** 7 motorcycles in a narrow frame causes rider bounding boxes to bleed into adjacent vehicle zones. Annotation text overlaps; over-counting occurs.

(b) **Pedestrian crowd + occlusion.** Bystanders near stopped bikes partially satisfy the foot-zone filter, inflating rider counts. Occluded heads receive **unknown** status, causing false negatives.

(c) **Riders sitting very close together.** When 2–3 riders are tightly packed, their bounding boxes merge. The head-crop classifier receives a mixed region, marking one rider **unknown** and missing the actual violation.

Figure 3: Failure cases: dense/packed scenes and closely-seated riders are the primary sources of false negatives and missed violations.

Identified Failure Modes

- F1. Dense pedestrian crowds.** When pedestrians stand very close to a parked or stopped motorcycle, their bounding boxes may satisfy both the foot-zone and overlap conditions, causing false triple-riding alerts. The height-consistency filter mitigates this but cannot fully resolve it when pedestrian and rider heights are similar.
- F2. Severely packed multi-bike frames.** In scenes with 5+ motorcycles within a narrow field-of-view, the spatial proximity of adjacent bikes causes rider bounding boxes to overlap multiple vehicle zones. The greedy distance-based scoring can assign the same rider to the wrong bike.
- F3. Blurred or oblique plates.** When a license plate is captured at an angle greater than $\sim 60^\circ$, or is motion-blurred, PaddleOCR’s angle classifier and recognizer may produce garbled output even after CLAHE enhancement.
- F4. Fully occluded helmet.** When a pillion rider’s head is completely hidden behind the driver, the head crop is uninformative. The fallback full-body crop also fails. The rider is conservatively marked **unknown** (no violation), risking a false negative.

6 Implementation Notes

6.1 Evaluator Interface Compliance

The `TrafficViolationDetector` class strictly follows the mandatory interface:

- All model loading occurs inside `__init__` exactly once.
- `predict(image_path: str) -> dict` is fully stateless — no instance variables are written during inference.
- The return dictionary matches the specification exactly.
- If the image file cannot be read, `predict()` returns `{"violations": []}` gracefully rather than raising an exception.

6.2 Offline Execution

PaddleOCR attempts to download model weights from the internet on first use by default. This would cause a crash under the evaluator’s network-isolated environment. To prevent this, all three PaddleOCR sub-models are manually cached inside `models/paddleocr_weights/` and their paths are explicitly passed to the `PaddleOCR` constructor via `det_model_dir`, `rec_model_dir`, and `cls_model_dir`.

6.3 Pinned Dependencies

`requirements.txt` pins all critical versions to ensure reproducibility:

Package	Pinned Version
ultralytics	==8.4.50
paddleocr	==2.7.3
paddlepaddle	==2.6.2
opencv-python	>=4.8.0
numpy	<2.0.0

Table 2: Pinned dependency versions in `requirements.txt`.

`numpy < 2.0.0` is required because PaddlePaddle 2.6.x C extensions are incompatible with NumPy 2.x’s changed ABI.

6.4 Model Discovery

The constructor searches for model files in a configurable list of root directories, making the submission resilient to the evaluator running from a different working directory. If `model_dir` is not specified, the default `./models/` relative to the script is used.

7 Conclusion

This submission delivers a robust, modular, and fully offline traffic violation detection pipeline that correctly identifies helmet violations, triple-riding incidents, and license plate numbers from arbitrary street images — all within the 250 MB model budget.

Key engineering contributions beyond a naive baseline:

- **Pseudo-depth foot-zone spatial filter** that eliminates bystanders without requiring a depth model.
- **Head-ROI helmet classification** that reduces classifier input noise.
- **High-recall custom plate detector** with CLAHE preprocessing for difficult plates.
- **IND/ND hologram artefact stripping** and position-aware OCR correction specific to Indian number plates.
- **Conservative unknown-rider policy** that keeps precision high.

The identified failure modes (densely packed scenes and pedestrian-rich environments) represent inherent limitations of single-frame 2D analysis without explicit depth estimation. We evaluated MiDaS v2.1-small as a depth cue but found its ≈ 105 MB weight cost outweighed the marginal accuracy gain under the 250 MB budget. The geometric pseudo-depth filter provides comparable benefit for free. Temporal tracking across video frames remains the most promising direction for eliminating remaining false associations.